



Figure 1: (Eyevine, 2023)

Canada Wildfire season 2023

An Interactive Fire Impact Map

University of Zurich || SDS210

Aroha Russi

Project Report

1. Introduction

The year 2023 marks the most destructive wildfire season on record for Canada, with fires burning more than 15 million of hectares from British Columbia to Quebec (Government of Canada, 2023). Large wildfires alter the air and drive a positive climate-fire feedback loop since the fire emissions contribute to hotter and drier weather conditions which then increases the frequency and spatial extent of fires (Government of Canada, 2013). The project addresses the following research question: ***Which Canadian Cities were exposed to the most risk during wildfire season in 2023?*** The aim is to quantify fire proximity to major cities and visualize the historical fire data in an interactive map. Additionally, live satellite fire detections from NASA FIRMS are integrated into the map as an extra layer to provide a current state of wildfires in Canada. (FIRMS US/CANADA, n.d.) The map then can be used as a support for risk communication and awareness.

2. Workflow and Implementation

The project is structured in three notebooks which should be executed in a specific order. The first notebook (*fire2023_data_cleaning.ipynb*) that has to be run loads in the first place the raw National Fire Database (NFDB) point dataset (Government of Canada, n.d.). Then in different steps it data cleaning steps were implemented like filtering for uncontrolled fires in 2023, decoding abbreviations for province and cause, removing spatial outliers with a longitude validity check. Then it classifies each fire record into five size classes (small, medium, big, very big and mega fire). This classification uses CIFFC Glossary of Terms classification system with changes to represent best the fire records and giving them a name not just numbers or letters (Grahame Gordon Wildfire, 2014). This classification is then used to assign for each class a weight. Since there are a lot of small fires, I decided to assign exponentially spaced weights (1,2,4,16,256) since with that the map looked also the best on every zoom level.

The second notebook (*cities_canada.ipynb*) aims to build a representative set of Canadian cities from the raw Simple Maps World Cities dataset (simplemaps, 2025). As seed locations all provincial and the federal capital are included. All other cities are candidates of being added if the requirements are met. Those requirements are that they have a high amount of population and additionally are at least 150km from every already selected city. This is to prevent overrepresentation of dense urban agglomerations such as in the Toronto area. The aim is to get a spatially balanced selection of the most populated cities of Canada.

The third notebook (*visualization.ipynb*) brings the first two notebooks together. Here a 200km buffer (Jordahl et al., 2020) is drawn around each selected city. Therefore the data needs to be in a metric projection EPSG: 3347 (Statistics Canada Lambert) (epsg.io, 2010). A spatial join (Jordahl et al., 2020) then aggregates the class weights of all fires within each buffer into a fire score. After merging to one layer (Jordahl et al., 2020) then the cities are classified into five impact levels, from no impact to very high impact and then visualized as Circle Markers on a leaflet map and customized colored from light (no impact) to dark (very high impact) (w3schools, n.d.). A leafmap is necessary to add a legend to the map. (Jordahl et al., 2020) Secondly a layer was added using an inferno inspired gradient to show all NFDB fire records of 2023 (Stack Overflow, 2017). A third layer fetches live Near Real Time fire detections from the NASA FIRMS VIIRS NOAA-20 API (FIRMS US/CANADA, n.d.).

3. Results

The interactive map reveals a clear northwest to southeast trend. There are two main clusters on the map: a large and intense zone from British Columbia, Alberta and Northwest Territories and the second cluster across Quebec and the Atlantic Canada in the south-east. What is clearly visible is, that the Central Canada, in particular Saskatchewan and Manitoba show a lower fire activity.

The impact scores and their colors reflect this spatial pattern. Cities in western Canada display High and Very High Impact scores, since they are within or adjacent to the most active fire zones. In the centre of Canada along the southern Prairie cities show mostly low or no impact which is being reflected also in the low density in the heat map.

Overall, the map demonstrates that the 2023 wildfire season was disproportionately severe in eastern and western parts of Canadian cities. As a limitation I would say that the map doesn't take into account the smoke dispersion or wind direction, which may make us underestimate the actual impact.

The live NASA FIRMS layer shows that at the moment there are not that many fires recorded for Canada and is therefore rather empty. But it also confirms that the 2023 season was extraordinary.

4. Challenges and Reflection

It was a bit challenging to figure out which operation requires a metric projection and which geographic one. When computing the impact scores with buffers calculated in degrees an error occurred which had me change this with a short code. This reinforced the value of defensive coding, as a single mismatch in projection can silently corrupt all calculations below without raising an error.

Another mentionable challenge was for sure selecting the relevant cities with an algorithm. A first approach just showed the additional to the primary cities the most populated cities of Canada. In Canada a lot of agglomerations of big cities are as well more populated than most cities across the country. To have not just clusters of urban areas but rather a balanced spatial distribution I implemented a spacing-based algorithm to just visualize a city if the city lies at least 159km from every other already selected city. A problem arised now, that with such an algorithm some cities haven't been selected, like Vancouver or Montréal which are high populated and standalone cities but an already selected city is nearer than 150 km.

5. Repository

<https://github.com/ArohaRussi/sds210project.git>

6. AI Use Disclosure

Most theoretical parts come from the course website (<https://hendrikwulf.github.io/sds210-jb/>). As an adding I used the following website (<https://geopandas.org/>). And finally, a large language model (Claude.ai) was used to check spelling, improve code structure and readability, and help debug operations. All decisions, the research questions, about the workflow design and the final implementation were developed and validated by me. Before including a code it was tested and understood.

Sources

- epsg.io. (2010, March 30). *NAD83 / Statistics Canada Lambert—EPSG:3347*. <https://epsg.io>
- Eyevine. (2023, June 15). Canada's wildfires have burnt an area 16 times larger than normal. *The Economist*. <https://www.economist.com/the-americas/2023/06/15/canadas-wildfires-have-burnt-an-area-16-times-larger-than-normal>
- FIRMS US/CANADA. (n.d.). *NASA-FIRMS*. Retrieved May 21, 2026, from <https://firms.modaps.eosdis.nasa.gov/map/>
- Government of Canada. (n.d.). *CWFIS Data & Service Catalogue* [Dataset]. Retrieved May 22, 2026, from <https://cwfis.cfs.nrcan.gc.ca/en/catalogue/results/7355c08f-1590-41cc-a751-8856afaa5959>
- Government of Canada. (2013, July 9). *Climate change and wildland fire*. <https://natural-resources.canada.ca/forests-forestry/wildland-fires/climate-change-wildland-fire>
- Government of Canada. (2023, September 7). *Canada's record-breaking wildfires in 2023: A fiery wake-up call*. <https://natural-resources.canada.ca/stories/simply-science/canada-s-record-breaking-wildfires-2023-fiery-wake-call>
- Grahame Gordon Wildfire. (2014). *Developing More Common Language, Terminology and Data Standards for Wildland Fire Management in Canada*.
- Jordahl, K., Bossche, J. V. den, Fleischmann, M., Wasserman, J., McBride, J., Gerard, J., Tratner, J., Perry, M., Badaracco, A. G., Farmer, C., Hjelle, G. A., Snow, A. D., Cochran, M., Gillies, S., Culbertson, L., Bartos, M., Eubank, N., maxalbert, Bilogur, A., ... Leblanc, F. (2020). *geopandas/geopandas: V0.8.1* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.3946761>
- simplemaps. (2025). *World Cities Database* [Dataset]. <https://simplemaps.com/data/world-cities>
- Stack Overflow. (2017, April 10). *Answer to "How to specify gradient parameter to folium heatmap?"* [Online post]. Stack Overflow. <https://stackoverflow.com/a/67838638>
- w3schools. (n.d.). *HTML Color Picker*. Retrieved May 21, 2026, from https://www.w3schools.com/colors/colors_picker.asp

Canadian Wildfires in 2023

NFDB points - Data Cleaning

This notebook cleans the National Fire Database (NFDB) point data. As a result one gets a filtered, classified dataset of uncontrolled wildfires in Canada for the year 2023.

Short summary of the following steps > 0. Import & Constants > 1. Preparation and Cleaning > 2. Filtering > 3. Classification of fire size > 4. Weighting > 5. Export of processed data

0. Import & Constants

```
from pathlib import Path
import pandas as pd

#With pathlib the storage paths stay portable across any device
#Using ../ we go up one level (out of notebook folder) to reach the
#project root and then enter the data folder

DATA_DIR = Path("../data")
RAW_PATH = DATA_DIR / "raw" / "NFDB_point.csv"
PROCESSED_PATH = DATA_DIR / "processed" / "fire_2023_clean.csv"
```

a) Analysis parameters

```
TARGET_YEAR = 2023

#Canada lies entirely in the western hemisphere,
#so any positive longitude is definitively a data entry error
MAX_VALID_LONGITUDE = 0
MIN_VALID_LATITUDE = 0

#Size class thresholds in hectares
# Minimum value observed: 200ha | Maximum value observed: 885388ha
SMALL_HA = 500
MEDIUM_HA = 1000
BIG_HA = 10000
VERY_BIG_HA = 100000

#Exponential weighting creates strong visual contrast between classes
CLASS_WEIGHTS = {
    "small": 1,
    "medium": 2,
    "big": 4,
    "very big": 16,
    "mega fire": 256
}

#preserving logical and not alphabetical order
CATEGORY_ORDER = ["small", "medium", "big", "very big", "mega fire"]
```

b) Load Raw Data

```
#You may have to take into account that when loading the data yourself  
#from the readme then the seperator may have been changed to a comma  
  
fire_df = pd.read_csv(RAW_PATH, sep=";")  
  
assert len(fire_df) > 0, "Loaded file is empty."  
print(f"Loaded {len(fire_df)} records from raw data.")
```

Loaded 15206 records from raw data.

1. Preparation and Cleaning

a) Renaming the columns to snake_case

```
#Only the columns relevant for later are renamed  
NEW_NAMES = {  
    "SRC_AGENCY": "province",  
    "LONGITUDE": "longitude",  
    "LATITUDE": "latitude",  
    "SIZE_HA": "size_ha",  
    "FIRE_ID": "fire_id",  
    "FID": "fid",  
    "CAUSE": "cause",  
    "YEAR": "year"  
}  
  
fire_df = fire_df.rename(columns=NEW_NAMES)
```

b) Dropping not used columns

```
COLUMNS_DROP = [  
    "the_geom", "NFDBFIREID", "NAT_PARK", "FIRENAME",  
    "REP_DATE", "OUT_DATE", "FIRE_TYPE", "RESPONSE",  
    "PROTZONE", "MORE_INFO", "MONTH", "DAY"  
]  
  
#Using errors="ignore" is intentional because if a columns is already absent  
#for example when re-running this cell, we do not want to that it crashes.  
  
fire_df = fire_df.drop(  
    columns = COLUMNS_DROP,  
    errors="ignore"  
) .copy()
```

c) Decoding Province Abbreviations

```
PROVINCE_FULL = {
    "ON": "Ontario",
    "BC": "British Columbia",
    "QC": "Quebec",
    "AB": "Alberta",
    "MB": "Manitoba",
    "SK": "Saskatchewan",
    "NS": "Nova Scotia",
    "NB": "New Brunswick",
    "NL": "Newfoundland and Labrador",
    "PE": "Prince Edward Island",
    "YT": "Yukon",
    "NT": "Northwest Territories",
    "NU": "Nunavut"}

#Ersetzen der Abkürzungen mit dem vollen Namen.
fire_df["province"] = fire_df["province"].map(
    PROVINCE_FULL).fillna(fire_df["province"])
```

d) Decoding Cause Abbreviations

```
CAUSE_FULL = {
    "H": "Human",
    "H-PB": "Prescribed Burn",
    "U": "Unknown",
    "N": "Natural"}

#Ersetzen der Abkürzungen mit dem vollen Namen.
fire_df["cause"] = fire_df["cause"].map(CAUSE_FULL).fillna(fire_df["cause"])
```

2. Filtering

```
# Prescribed burns are controlled fire events
# This analysis focuses on uncontrolled fires only.

drop_rows = fire_df[fire_df["cause"]=="Prescribed Burn"].index

fire_df = fire_df.drop(index = drop_rows)
```

```
#Isolating just the target year (TARGET_YEAR)
fire_2023 = fire_df[fire_df["year"]== TARGET_YEAR]
```

```

#Remove spatial outliers;
#positive longitudes and negative latitudes are impossible for Canada

fire_2023 = fire_2023[fire_2023["longitude"] <= MAX_VALID_LONGITUDE]
fire_2023 = fire_2023[fire_2023["latitude"] >= MIN_VALID_LATITUDE]

print(f"There were {len(fire_2023)} uncontrolled fires starting in 2023")

```

There were 964 uncontrolled fires starting in 2023

```

fire_2023 = fire_2023.sort_values(by="size_ha", ascending=False)

```

3. Classification of fire size

```

def classify_size(size_ha):
    """
    Classifies a wildfire event by its burned area

    Thresholds follow partly the Canadian wildfire size classification system.
    But since 2023 was a severe year they were a bit adapted so that the count
    of fires in each class is well distributed.

    As Thresholds the constants from Step 0 (SMALL_HA, MEDIUM_HA,
    BIG_HA, and VERY_BIG_HA) will be used

    Parameters
    -----
    size_ha: Burned area in hectares

    Returns
    -----
    One of: "small", "medium", "big", "very big", "mega fire".
    """
    if size_ha <= SMALL_HA:
        return "small"
    elif 500 <= size_ha < MEDIUM_HA:
        return "medium"
    elif 1000 <= size_ha < BIG_HA:
        return "big"
    elif 10000 <= size_ha < VERY_BIG_HA:
        return "very big"
    else:
        return "mega fire"

# Application of classification
fire_2023["size_class"] = fire_2023["size_ha"].apply(classify_size)

```

```
#Use an ordered Categorical so for later legends respect the logically set up  
#sequence and noth sorting it alphabetically
```

```
fire_2023["size_class"] = pd.Categorical(  
    fire_2023["size_class"],  
    categories=CATEGORY_ORDER,  
    ordered=True  
)  
  
print(fire_2023["size_class"].value_counts().sort_index())
```

```
size_class  
small      203  
medium     131  
big        370  
very big   224  
mega fire   36  
Name: count, dtype: int64
```

4. Weighting

```
#Adding predefined class weights of Step 0
```

```
fire_2023["class_weight"] = fire_2023["size_class"].map(CLASS_WEIGHTS)
```

5. Export of processed data

```
fire_2023.to_csv(PROCESSED_PATH, index=False)  
  
print(f"Saved {len(fire_2023):} records to {PROCESSED_PATH}")
```

```
Saved 964 records to ..\data\processed\fire_2023_clean.csv
```

You have now successfully cleaned the data and can proceed with the next notebook about Canadas most populated cities.

Canadian Wildfires in 2023

Selection of relevant Canadian Cities

This notebook selects a representative set of Canadian cities out of the World Cities dataset. Of each province the capital is being included (admin & primary). To fill up the 40 relevant cities an algorithm was computed to add the most populated cities of Canada. Since in Canada there is a huge difference between high populated and a low populated City an additional algorithm was included to only select a city if its at least 150km apart from any other selected city

Short summary of the following steps

0. Import & Constants
1. Preparation and Cleaning
2. City Selection
3. Export of processed data

0. Import & Constants

```
from pathlib import Path

import pandas as pd
import geopandas as gpd

#With pathlib the storage paths stay portable across any device
#Using ../ we go up one level (out of notebook folder) to reach the
#project root and then enter the data folder

DATA_DIR = Path("../data")
RAW_PATH = DATA_DIR / "raw" / "worldcities.csv"
PROCESSED_PATH = DATA_DIR / "processed" / "selected_cities_canada.csv"
```

a) Analysis parameters

```
#Coordinate reference systems
#EPSG: 4326 is being used as a default in the base data (geographic CRS (in degrees))
#EPSG: 3347 is the metric projection for Canada, required for calculations in meters

CRS_DEFAULT_DEGREE = "EPSG: 4326"
CRS_COUNTRY_METRIC = "EPSG: 3347"

#storing the relevant City as a constante
TARGET_COUNTRY = "Canada"

#Cities which are always being included regardless of spacing
CAPITAL_CITIES = ["primary", "admin"]
```

```

#Number of Cities which have to be selected
MAX_CITIES = 40

#Minimum allowed distance between cities in meters.
#Any other City closer than this distance is skipped to avoid spatial clustering

MIN_SPACE_KM = 150

```

b) Load raw data

```

cities_df = pd.read_csv(RAW_PATH, sep=",")

assert len(cities_df) > 0, "Loaded file is empty."
print(f"Loaded {len(cities_df)} cities worldwide from raw data.")

```

Loaded 48059 cities worldwide from raw data.

1. Preparation and Cleaning

a) Filter

```

#filtering to target country

cities_country_df = cities_df[
    cities_df["country"] == TARGET_COUNTRY].copy()

assert len(cities_country_df) > 0, f"No cities found in '{TARGET_COUNTRY}'."
print(f"{TARGET_COUNTRY} cities in raw data: {len(cities_country_df)}")

```

Canada cities in raw data: 485

b) CRS transformation

```

#transforming the data into a GeoDataFrame in geographic CRS
#Using panda we need a transformation because otherwise latitude and
#longitude are simple numbers and not recognized as a geometry

cities_country_gdf = gpd.GeoDataFrame(
    cities_country_df,
    geometry=gpd.points_from_xy(
        cities_country_df["lng"],
        cities_country_df["lat"]),
    crs = CRS_DEFAULT_DEGREE
)

#For a metric calculation such as a distance the data needs to be

```

```

#stored in a metric crs (CRS_COUNTRY_METRIC)
#If you change the "TARGET_COUNTRY" you have to change this
#CRS_COUNTRY_METRIC as well

cities_country_gdf = cities_country_gdf.to_crs(CRS_COUNTRY_METRIC)

assert cities_country_gdf.crs is not None, "Reprojection failed"
print(f"GeoDataFrame CRS: {cities_country_gdf.crs.to_epsg()}")

```

GeoDataFrame CRS: 3347

2. City selection

a) Preparation of inputs

```

#Capital and admin cities are always included no matter the population.
primary_cities = cities_country_gdf[
    cities_country_gdf["capital"].isin(CAPITAL_CITIES)]

#All the others are possible candidates for the spacing-based selection
other_cities = cities_country_gdf[
    ~cities_country_gdf.index.isin(primary_cities.index)]

#By sorting the cities we make sure, that the most populated places
#fill the open slots in the list.
other_cities = other_cities.sort_values(
    "population",
    ascending=False
).reset_index(drop=True)

```

b) Definition Selection Loop

```

def select_cities_by_space(
    seed_gdf: gpd.GeoDataFrame,
    candidates_gdf: gpd.GeoDataFrame,
    min_spacing_m: float,
    max_cities: int,
) -> gpd.GeoDataFrame:
    """
    Adds possible cities to a predefined selection
    of cities if they are spaced sufficiently.

    It iterates through candidates with the largest population first.
    Then it includes the city only when the distance to each already
    selected city is greater than min_spacing_m. This assumption was
    made to prevent that multiple urban agglomeration from the same
    city are being selected. Finally it stops when max_cities is
    reached so that not every city in the country is on the final map.
    """

```

```

Parameters:
-----
seed_gdf: gpd.GeoDataFrame
    In this case the Capital city and all admin cities that are
    always included.

candidates_gdf: gpd.GeoDataFrame
    The remaining candidating cities to evaluated, sorted by
    descending population.

min_spacing_m: float
    Minimum distance in meters between two selected cities.

max_cities: int
    Upper limit on the wanted number of selected cities.

Returns
-----
gpd.GeoDataFrame
    A combined GeoDataFrame of predefined cities and the other
    valid candidating cities.
"""

selected_cities_gdf = seed_gdf.copy()

for index, city in candidates_gdf.iterrows():
    distances = selected_cities_gdf.distance(city.geometry)
    min_distance = distances.min()

    if min_distance >= min_spacing_m:
        selected_cities_gdf = pd.concat(
            [selected_cities_gdf, city.to_frame().T],
            ignore_index=True)

    if len(selected_cities_gdf) == MAX_CITIES:
        break

return selected_cities_gdf

```

c) Running the Selection Loop

```
selected_cities_gdf = select_cities_by_space(
    seed_gdf = primary_cities,
    candidates_gdf = other_cities,
    min_spacing_m = MIN_SPACE_KM * 1000,
    max_cities = MAX_CITIES,
)

assert len(selected_cities_gdf) > 0, "The dataset is empty."
print(f"Total cities selected: {len(selected_cities_gdf)}")
```

Total cities selected: 40

3. Export of processed Data

```
selected_cities_gdf.to_csv(PROCESSED_PATH, index=False)

print(f"Saved {len(selected_cities_gdf)} records to {PROCESSED_PATH}")
```

Saved 40 records to ..\data\processed\selected_cities_canada.csv

Now all relevant Cities for the analysis are selected and can be visualized. Therefore open now the visualization.ipynb notebook and follow the code to combine the fire data and the cities in an interactive map

Canadian Wildfires in 2023

Visualization of the data in an interactive Map

This notebook produces an interactive Folium visualization of the recorded uncontrolled wildfire events in Canada in 2023 and their impact on the most relevant cities in Canada.

Therefore it combines three togglable layers * **Heatmap** with all uncontrolled NFDB fires in 2023, weighted by size class

- **Cities as CircleMarkers** coloured by their fire-impact score
- **Live heatmap** of recent active fires retrieved from the NASA FIRMS API

Short summary of the following steps

0. Import & Constants
1. Computation of GeoDataFrames & CRS Reprojection
2. Spatial Join & Fire Scores for Cities
3. Classification of cities impact levels
4. Live fire data from NASA FIRMS
5. Put together the map layers
6. Export of the final map

IMPORTANT!! an active internet connection is necessary to run this notebook, since otherwise the NASA FIRMS data can't be loaded.

0. Import & Constants

```
import requests
from datetime import date
from pathlib import Path

import geopandas as gpd
import pandas as pd
import folium
from folium import plugins
import leafmap.foliummap as leafmap

DATA_DIR = Path("../data")

FIRE_CSV_PATH = DATA_DIR / "processed" / "fire_2023_clean.csv"
FIRE_GEOJSON_PATH = DATA_DIR / "processed" / "fire_2023_points.geojson"

CITIES_CSV_PATH = DATA_DIR / "processed" / "selected_cities_canada.csv"
CITIES_GEOJSON_PATH = DATA_DIR / "processed" / "selected_cities_canada.geojson"

LIVE_FIRE_PATH = DATA_DIR / "processed" / "fire_live.csv"

MAP_OUTPUT_PATH = Path("../outputs") / "interactive_firemap.html"
```

a) Analysis parameters

```
# EPSG:4326 in geographic degrees; required by Folium and for GeoJSON export.
# EPSG:3347 in metres; required for buffer & spatial join.

CRS_DEFAULT_DEGREE = "EPSG: 4326"
CRS_COUNTRY_METRIC = "EPSG: 3347"

#Storing the symbology characteristics to change it just once for both heatmaps
#HM means HeatMap.

HM_RADIUS = 13
HM_BLUR = 8
HM_MIN_OPACITY = 0.35

#inferno-inspired colour ramp, where dark > light means low > high intensity.
HM_GRADIENT = {
    0.00: "#000004",
    0.20: "#420A68",
    0.40: "#6A176E",
    0.60: "#932667",
    0.80: "#DD513A",
    1.00: "#FCFFA4",
}

#Later cities accumulate a fire score from number of fires within this radius
BUFFER_KM = 200

#Setup of fire_score thresholds which define for each city the impact level.
#S therefore means Score
S_LOW = 20
S_MODERATE = 50
S_HIGH = 256

#Essential variables for NASA FIRMS live fire API data from VIIRS NOAA-20 NRT data
FIRMS_MAPKEY = "b73850f35cfea7f219a3969b6dfd1d9a6"

CANADA_BBOX = {"north": 73.2267, "south": 44.6401, "west": -140.625, "east": -49.2188}

#For layering the three toggleable layers we need to set up first a basemap for Canada.
MAP_CENTER = [58.03,-90.82]
ZOOM_START = 4
TILE_STYLE = "CartoDB.DarkMatterNoLabels"
```

b) Load Processed Data

```
fire_2023 = pd.read_csv(FIRE_CSV_PATH)
cities_df = pd.read_csv(CITIES_CSV_PATH)

assert len(fire_2023) > 0, "Failed; you may re-run the data cleaning notebook."
```

```
assert len(cities_df) > 0, "Failed; you may re-run the city selection notebook."

print(f"Loaded {len(fire_2023)} wildfire events and {len(cities_df)} cities of Canada")
```

Loaded 964 wildfire events and 40 cities of Canada

1. Computation of GeoDataFrames & CRS Reprojection

```
#For visualization in Folium we need the data in a GeoDataFrame and as a GeoJSON file
fire_2023_gdf = gpd.GeoDataFrame(
    fire_2023, geometry=gpd.points_from_xy(
        fire_2023.longitude, fire_2023.latitude),
    crs= CRS_DEFAULT_DEGREE)

fire_2023_gdf.to_file(FIRE_GEOJSON_PATH, driver="GeoJSON")

selected_cities_gdf = gpd.GeoDataFrame(
    cities_df,
    geometry=gpd.points_from_xy(
        cities_df.lng,
        cities_df.lat),
    crs= CRS_DEFAULT_DEGREE)

selected_cities_gdf.to_file(CITIES_GEOJSON_PATH, driver="GeoJSON")

#For operations based on distance metric projection is required
cities_metric_gdf = selected_cities_gdf.to_crs(CRS_COUNTRY_METRIC)
fire_2023_metric_gdf = fire_2023_gdf.to_crs(CRS_COUNTRY_METRIC)
```

2. Spatial Join & Fire Scores for Cities

a) Setting up buffers around cities

```
#Buffer the city to then count the fires inside it for impact score
cities_buffered = cities_metric_gdf.copy()
cities_buffered["geometry"] = cities_buffered.geometry.buffer(
    BUFFER_KM * 1000)
```

b) Spatial Join

```
#A Spatial Join is needed to find all fires inside the buffer
#Since we want to have also cities with no fires within their buffer
#we make a left join.

joined_data = gpd.sjoin(
    cities_buffered,
    fire_2023_metric_gdf,
```

```

    how="left", #cities without fires should be visible
    predicate = "contains"
)

```

c) Aggregation fire score per city

```

#Aggregating fire scores per city to gain an impact score.
#The class_weight was set up to consider the severity.

fire_per_city = (
    joined_data
    .groupby("city")["class_weight"]
    .sum()
    .reset_index()
    .rename(columns={"class_weight": "fire_score"})
)

#Merging the scores back gives us all relevant data together and by
#converting it into the geographic CRS it can be later visualized.

selected_cities_gdf = (
    cities_metric_gdf
    .merge(fire_per_city, on="city", how="left")
    .to_crs(CRS_DEFAULT_DEGREE)
)

#For replacing the NaNs (Cities with no fires in their buffer)
#we fill it up with a score of 0

selected_cities_gdf["fire_score"] = selected_cities_gdf[
    "fire_score"].fillna(0)

```

3. Classification of cities impact levels

```

def impact(fire_score):
    """
    Classifies the cumulative fire score of a city into wildfire impact

    The thresholds are defined by the constants SCORE_LOW,
    SCORE_MODERATE, and SCORE_HIGH. Those are the sums of fire class_weight
    values (1 / 2 / 4 / 16 / 256) from the NFDB cleaning step.

    Parameters
    -----
    fire_score_1
        Cumulative class_weight of all fires within CITY_BUFFER_M of a city.

    Returns
    -----
        One of: "No impact", "Low impact", "Moderate impact", "High impact", "Very high impact".
    """

```

```

"""
if fire_score == 0:
    return "No impact"
if fire_score < S_LOW:
    return "Low impact"
if fire_score < S_MODERATE:
    return "Moderate impact"
if fire_score < S_HIGH:
    return "High impact"
return "Very high impact"

selected_cities_gdf["impact"] = selected_cities_gdf[
    "fire_score"].apply(impact)

print(selected_cities_gdf["impact"].value_counts())

```

```

impact
No impact      13
Low impact     11
Very high impact  6
Moderate impact  6
High impact     4
Name: count, dtype: int64

```

4. Live fire data from NASA FIRMS

```

today = date.today()
api_url = f"https://firms.modaps.eosdis.nasa.gov/usfs/api/area/csv/{FIRMS_MAPKEY}/VIIRS_NOAA20_NRT/world"

response = requests.get(api_url)

if response.status_code == 200:
    fire_live_df = pd.read_csv(api_url)
    fire_live_df.to_csv(LIVE_FIRE_PATH, index=False)

    fire_live_df = fire_live_df[
        (fire_live_df["latitude"] >= CANADA_BBOX["south"]) &
        (fire_live_df["latitude"] <= CANADA_BBOX["north"]) &
        (fire_live_df["longitude"] >= CANADA_BBOX["west"]) &
        (fire_live_df["longitude"] <= CANADA_BBOX["east"])
    ]
    print(f"Live fire records for Canada: {len(fire_live_df)}")
else:
    print("Error")

```

Live fire records for Canada: 32

5. Put together the map layers

a) Base map

```
#By using leafmap we make sure that we can add a legend.
canada_map = leafmap.Map(
    location = MAP_CENTER,
    zoom_start = ZOOM_START,
    tiles = TILE_STYLE,
)
```

b) Weighted Heat Map for wildfire events in 2023

```
#Heatmap for weighted fires
heat_data = [[point.xy[1][0], point.xy[0][0], weight]
              for point, weight in zip (
                  fire_2023_gdf.geometry,
                  fire_2023_gdf["class_weight"])
              ]

plugins.HeatMap(
    heat_data,
    radius=HM_RADIUS,
    blur=HM_BLUR,
    min_opacity=HM_MIN_OPACITY,
    name = "Wildfires in 2023",
    gradient= HM_GRADIENT
).add_to(canada_map)
```

```
<folium.plugins.heat_map.HeatMap at 0x232affd3b60>
```

c) Visualization of impact on selected cities

```
impacted_cities_layer = folium.FeatureGroup(name = "Impacted Cities in 2023")

for city,row in selected_cities_gdf.iterrows():
    fire_score = row["fire_score"]
    if fire_score == 0:
        city_color = "#E0FFFF"
        city_radius = 4

    elif fire_score > 0 and fire_score < 20:
        city_color = "#87CEFA"
        city_radius = 5

    elif fire_score >= 20 and fire_score < 50:
        city_color = "#1E90FF"
        city_radius = 7
```

```

elif fire_score >= 50 and fire_score < 256:
    city_color = "#0000FF"
    city_radius = 8

else:
    city_color = "#191970"
    city_radius = 10

folium.CircleMarker(
    location=[row.geometry.y, row.geometry.x],
    radius = city_radius,
    color = "gray",
    weight = 0.9,
    fill = True,
    fill_color=city_color,
    fill_opacity = 0.95,
    tooltip= (f"Name: {row['city']} <br>"
              f"Country: {row['country']}<br>"
              f"Population: {row['population']} <br>"
              f"Impact: {row['impact']}"),
    ).add_to(impacted_cities_layer),

impacted_cities_layer.add_to(canada_map)

```

<folium.map.FeatureGroup at 0x232b00fd6a0>

d) HeatMap for NASA FIRMS recent data

```

live_heatmap = [
    [row["latitude"], row["longitude"]]
    for _, row in fire_live_df.iterrows()
]

plugins.HeatMap(
    live_heatmap,
    radius=HM_RADIUS,
    blur=HM_BLUR,
    min_opacity=HM_MIN_OPACITY,
    name = "Live wildfire",
    show = False,
    gradient= HM_GRADIENT
).add_to(canada_map)

```

<folium.plugins.heat_map.HeatMap at 0x232b002b250>

e) Legend and Layer control

```

legend_impact ={
    "No Impact": "#E0FFFF",

```

```

    "Low Impact": "#87CEFA",
    "Medium Impact": "#1E90FF",
    "High Impact": "#0000FF",
    "Very High Impact": "#191970"
}

canada_map.add_legend(title="Fire impact on city",
                      legend_dict=legend_impact)

folium.LayerControl(collapsed=False).add_to(canada_map)

```

```
<folium.map.LayerControl at 0x232b1714980>
```

6. Export of the final map

```

canada_map.save(MAP_OUTPUT_PATH)

print(f"Map saved to: {MAP_OUTPUT_PATH}")

```

```
Map saved to: ..\outputs\interactive_firemap.html
```

Display Map

```

#Visualization of Impact Map for Wildfire season 2023 with togglable layers
canada_map

```